

SYNTAXE SQL

MINIMUM VITAL

SELECTION DE DONNEES

SELECT [**ALL** | **DISTINCT**]

{ * | [*utilisateur.*] {*table* | *vue* }. * | *exp* }

[, { [*utilisateur.*] {*table* | *vue* }. * | *exp* }] ...

FROM [*utilisateur.*] {*table* | *vue* }

[**WHERE** *condition*]

[**GROUP BY** *exp* [, *exp*] ... [**HAVING** *condition*]]

[{ **UNION** | **INTERSECT** | **MINUS** } *question*]

[**ORDER BY** { *exp* | *position* } [**ASC** | **DESC**]
[, { *exp* | *position* } [**ASC** | **DESC**]] ...

<u>ALL</u>	toutes les lignes sont sélectionnées (vrai par défaut).
DISTINCT	les lignes en double sont supprimées.
*	précise que toutes les colonnes de toutes tables sont écrites.
<i>table</i> .*	précise que toutes les colonnes de la table sont écrites.
<i>position</i>	est un entier qui désigne la position de la colonne à partir de laquelle est
effectué	le classement.
<u>ASC</u> DESC	ordre ascendant ou descendant.

Remarques :

1) les clauses **CONNECT BY**, **START WITH**, **GROUP BY** et **HAVING** peuvent être données dans n'importe quel ordre entre elles.

2) Utilisation de **GROUP BY** et **HAVING**

La clause **GROUP BY** permet de grouper les lignes sélectionnées tandis que la clause **HAVING** limite les groupes sélectionnés.

Exemple : "Donner les noms et la capacité totale (somme des capacités) par type d'avion de marque airbus (a300, a320, ...) ?"

```
SELECT nomav, sum(cap)
FROM avion
WHERE nomav LIKE 'a%'
GROUP BY nomav ;
```

3) Utilisation de **ORDER BY**

La clause **ORDER BY** permet de trier les lignes de résultat suivant des expressions ou le numéro des colonnes.

Table / **Column** / **Row** / **value** / **Primary key** / **Foreign key**

empno	name	job	mgr	hiredate	sal	comm	deptno
7369	SMITH	CLERK	7902	1980-12-17	800		20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-04-02	2975		20
7654	MARTIN	SALESMAN	7698	1981-09-28	1250	1400	30
7698	BLAKE	MANAGER	7839	1981-05-01	2850		30
7782	CLARK	MANAGER	7839	1981-06-09	2450		10
7788	SCOTT	ANALYST	7566	1982-12-09	3000		20
7839	KING	PRESIDENT		1981-11-17	5000		10
7844	TURNER	SALESMAN	7698	1981-09-08	1500	0	30
7876	ADAMS	CLERK	7788	1983-01-12	1100		20
7900	JAMES	CLERK	7698	1981-12-03	950		30
7902	FORD	ANALYST	7566	1981-12-03	3000		20
7934	MILLER	CLERK	7782	1982-01-23	1300		10

MODIFICATION DE DONNEES

1 - MODIFICATION DE LIGNES

```
UPDATE [utilisateur.] {table | vue }  
    SET colonne = exp [colonne = exp ] ...  
    WHERE condition
```

ou bien

```
UPDATE [utilisateur.] {table | vue }  
    SET (colonne [,colonne ] ... ) = (question )  
        [ , (colonne [,colonne ] ... ) = (question ) ] ...  
    WHERE condition
```

Exemple : "Remplacer la capacité des atr par leur valeur ancienne diminuée de 10."

UPDATE avion

SET cap = cap + 10

WHERE nomav = 'atr' ;

2 - SUPPRESSION DE LIGNES

```
DELETE FROM [utilisateur.] {table | vue }  
    WHERE condition
```

Exemple : "Supprimer le pilote identifié par le numéro 5."

DELETE FROM pilote WHERE numpil = 5 ;

5 - modification des objets

CREATION DES TABLES

```
CREATE TABLE nom_table (  
    Nom_col1 type_donnees1 [contrainte],  
    nom_col2 type_donnees2 [contrainte],  
    ...  
    nom_coln type_donneesN [contrainte]  
    [tablecontrainte]);
```

Les contraintes pour chaque attributs peuvent être :

PRIMARY KEY, NOT NULL, FOREIGN KEY, CONSTRAINT (une contrainte)

MODIFICATION DES TABLES

```
ALTER TABLE [utilisateur.] table  
  
    [ ADD ( colonne-def [ ,colonne-def ] ... ) ]  
    [ MODIFY ( colonne-def [ ,colonne-def ] ... ) ]  
    [ DROP CONSTRAINT contrainte ]
```

ADD/MODIFY ajoute ou modifie des définitions de colonnes, des contraintes sur les colonnes, les valeurs par défaut sur les colonnes.

DROP supprime une contrainte sur une colonne ou sur une table (contraintes sur plusieurs colonnes).

Remarques :

Lorsqu'une colonne est ajoutée à une table, la valeur initiale pour chaque ligne est NULL.

Lorsqu'une vue a été définie au moyen d'une question sur toutes les colonnes d'une table (SELECT *), elle doit être supprimée puis recréée.

Exemple1 : "Ajouter une colonne salaire à la table pilote ; la valeur doit être supérieure 4000 F."

ALTER TABLE pilote

ADD (sal NUMBER(7,2) CHECK sal > 4000 CONSTRAINT sal_min) ;

Exemple2 : "Agrandir la colonne nompil et supprimer la contrainte sal_min sur la colonne salaire des pilotes."

ALTER TABLE pilote

MODIFY (nompil CHAR(20))

DROP CONSTRAINT sal_min ;

